

Data Structures in R & Python

Lecture 06

Dr. Colin Rundel

Python classes

Basic syntax

A `class` statement bundles attributes (data) and methods (functions) into a new type. Methods receive the instance as their first argument, conventionally named `self`.

```
1 class rect:
2     """An object representing a rectangle"""
3     p1 = (0, 0)
4     p2 = (1, 2)
5
6     def area(self):
7         return abs((self.p1[0] - self.p2[0]) *
8                     (self.p1[1] - self.p2[1]))
9
10    def set_p1(self, p1):
11        self.p1 = p1
```

```
1 x = rect()
2 x.area()
```

2

```
1 x.p1
```

(0, 0)

```
1 x.set_p1((1, 1))
2 x.area()
```

0

```
1 x.p2 = (3, 3)
2 x.area()
```

4

`__init__`

Class construction, i.e. calling the class `rect()`, creates a new instance of the class its `__init__()` method.

```
1 class rect:
2     """An object representing a rectangle"""
3
4     def __init__(self, p1=(0, 0), p2=(1, 1)):
5         self.p1 = p1
6         self.p2 = p2
7
8     def area(self):
9         return abs((self.p1[0] - self.p2[0]) *
10                    (self.p1[1] - self.p2[1]))
```

```
1 rect().area()
```

1

```
1 rect((0, 0), (3, 3)).area()
```

9

```
1 z = rect(p1=(-1, -1))
2 z.p1
```

(-1, -1)

```
1 z.p2
```

(1, 1)

Methods whose names start and end with double underscores (`__init__`, `__repr__`, ...) are called “dunder” methods -

Class vs instance attributes

Attributes assigned in the class body are class attributes shared by every instance (useful for constants), while attributes assigned via `self` are instance attributes belonging to a specific object.

```
1 class die:
2     sides = 6
3
4     def __init__(self, value=1):
5         self.value = value
```

```
1 d = die(3)
2 d.value
```

3

```
1 d.sides
```

6

```
1 die.sides
```

6

```
1 die.value
```

AttributeError: type object 'die' has no attribute 'value'

```
1 vars(d)
```

```
{'value': 3}
```

```
1 [m for m in dir(d) if not m.startswith("__")]
```

```
['sides', 'value']
```

```
1 die.sides = 3
2 d.sides
```

3

`dir()` lists all attributes and methods of an object (including inherited dunder methods), while `vars()` returns just the

Mutate or return?

Python methods like `sort()` and `append()` modify the object in place and return `None`, while functions like `sorted()` return a new object and leave the original alone. R functions essentially always do the latter, so results must be assigned back to be kept.

```
1 x = [3, 1, 2]
2 print( x.sort() )
```

py

None

```
1 x
```

py

[1, 2, 3]

```
1 x = [3, 1, 2]
2 sorted(x)
```

py

[1, 2, 3]

```
1 x
```

py

[3, 1, 2]

```
1 x = c(3, 1, 2)
2 sort(x)
```

R

[1] 1 2 3

```
1 x
```

R

[1] 3 1 2

```
1 x = sort(x)
2 x
```

R

[1] 1 2 3

Method chaining

Returning `self` (or a new object) from a method allows calls to be chained, with each method operating on the result of the previous one. This plays the same role in Python that the pipe does in R.

```
1 def set_p1(self, p1):  
2     self.p1 = p1  
3     return self  
4  
5 rect.set_p1 = set_p1
```

py

```
1 ( rect()  
2   .set_p1((-1, -1))  
3   .area()  
4 )
```

py

4

```
1 " Hello World ".strip().lower().split()
```

py

```
['hello', 'world']
```

Chaining in libraries like pandas works the same way, though most of those methods return a new object rather than

__str__ and __repr__

Every object has a default string representation, which can be overridden these dunder methods:

```
1 rect()
```

```
<__main__.rect object at 0x1122ef5c0>
```

```
1 print(rect())
```

```
<__main__.rect object at 0x11233f2f0>
```

- `__repr__()` - returns an unambiguous representation (ideally valid Python).
- `__str__()` - used by `print()` and `str()`, falls back to `__repr__()`.

```
1 def rect_repr(self):  
2     return f"rect({self.p1}, {self.p2})"  
3  
4 rect.__repr__ = rect_repr
```

```
1 rect()
```

```
rect((0, 0), (1, 1))
```

```
1 print(rect())
```

```
rect((0, 0), (1, 1))
```

```
1 [rect(), rect((1, 1), (2, 2))]
```

```
[rect((0, 0), (1, 1)), rect((1, 1), (2, 2))]
```

Other special methods

Operators and built-in functions also dispatch to special methods - this is the mechanism behind the operator overloading we've seen previously.

```
1 def rect_eq(self, other):  
2     return (self.p1 == other.p1 and  
3           self.p2 == other.p2)  
4  
5 rect.__eq__ = rect_eq  
6  
7 def rect_contains(self, pt):  
8     return all(  
9         min(a, b) <= x <= max(a, b)  
10        for a, b, x in zip(self.p1, self.p2, pt  
11    )  
12  
13 rect.__contains__ = rect_contains
```

```
1 rect()
```

```
rect((0, 0), (1, 1))
```

```
1 rect() == rect()
```

True

```
1 rect() == rect((1, 1), (2, 2))
```

False

```
1 (0.5, 0.5) in rect()
```

True

```
1 (2, 2) in rect()
```

False

Inheritance

A class can inherit from another class(es), gaining all of its attributes and methods. The subclass can add new methods and override existing ones - `super()` gives access to the parent's version.

```
1 class square(rect):
2     def __init__(self, p1=(0, 0), l=1):
3         if not isinstance(l, (int, float)):
4             raise TypeError("l must be a number")
5         self.l = l
6         super().__init__(p1, (p1[0] + l, p1[1] + l))
7
8     def __repr__(self):
9         return f"square({self.p1}, {self.l})"
```

```
1 s = square((1, 1), 2)
2 s
```

square((1, 1), 2)

```
1 s.p2
```

(3, 3)

```
1 s.area()
```

4

```
1 (2, 2) in s
```

True

```
1 square(l="a")
```

TypeError: l must be a number

isinstance() vs type()

`isinstance()` respects inheritance while `type()` reports the exact class - the former is almost always what you want when validating inputs, but be aware of what type inherits what.

```
1 isinstance(s, square)
```

py

True

```
1 isinstance(s, rect)
```

py

True

```
1 type(s) is rect
```

py

False

```
1 isinstance(3, (int, float))
```

py

True

```
1 isinstance("3", (int, float))
```

py

False

```
1 isinstance(True, int)
```

py

True

```
1 type(True) is int
```

py

False

```
1 bool.__mro__
```

py

(<class 'bool'>, <class 'int'>, <class 'object'>)

Tuples

Tuples

Python tuples are *heterogeneous, ordered, immutable* containers - lists whose contents cannot be changed. R has no tuple type, a list (or an atomic vector) is used instead.

```
1 (1, 2, 3)
```

py

```
(1, 2, 3)
```

```
1 (1, True, "abc")
```

py

```
(1, True, 'abc')
```

```
1 type( (1) )
```

py

```
<class 'int'>
```

```
1 type( (1,) )
```

py

```
<class 'tuple'>
```

```
1 x = (1, 2, 3)
```

py

```
2 x[2] = 5
```

TypeError: 'tuple' object does not support

```
1 x = (1, [2, 3])
```

py

```
2 x[1].append(4)
```

```
3 x
```

```
(1, [2, 3, 4])
```

It is the *comma* that creates a tuple, not the parentheses - `(1)` is just the number 1 and a one element tuple must be written `(1,)`.

Unpacking

Assigning a sequence to several names at once *unpacks* it. A name prefixed with *** collects any remaining values into a list.

```
1 def minmax(x):  
2     return min(x), max(x)  
3 minmax([3, 1, 2])
```

py

(1, 3)

```
1 lo, hi = minmax([3, 1, 2])  
2 print(lo, hi)
```

py

1 3

```
1 x, y = 1, 2  
2 x, y = y, x  
3 x, y
```

py

(2, 1)

```
1 first, *rest = range(6)  
2 first
```

py

0

```
1 rest
```

py

[1, 2, 3, 4, 5]

```
1 pairs = [("a", 1), ("b", 2)]  
2 for i, (k, v) in enumerate(pairs):  
3     print(i, k, v)
```

py

0 a 1
1 b 2

The number of names must match the length of the sequence (`x, y = [1, 2, 3]` is a `ValueError`) unless one of them is

Dictionaries

Dictionaries

Python `dicts` are *mutable* containers of `key: value` pairs, designed for the efficient lookup of a value by its key. The closest (common) R analog is a named list (or a named atomic vector).

```
1 {"abc": 123, "def": 456}
```

py

```
{'abc': 123, 'def': 456}
```

```
1 dict([("abc", 123), ("def", 456)])
```

py

```
{'abc': 123, 'def': 456}
```

```
1 dict(abc=123, hello=456)
```

py

```
{'abc': 123, 'hello': 456}
```

```
1 dict(zip(["abc", "def"], [1, 2]))
```

py

```
{'abc': 1, 'def': 2}
```

```
1 list(abc = 123, def = 456)
```

R

```
$abc  
[1] 123
```

```
$def  
[1] 456
```

```
1 c(abc = 123, def = 456)
```

R

```
abc def  
123 456
```

Since Python 3.7 dictionaries preserve insertion order. The keyword form of `dict()` only works for keys that are valid

Keys must be hashable

Dictionary keys and set elements must be hashable and values can be anything.

Hashable	Unhashable
<code>int, float, bool, complex</code>	<code>list</code>
<code>str, bytes, range</code>	<code>dict</code>
<code>None, frozenset</code>	<code>set</code>
<code>tuple</code> if all its elements are hashable	<code>tuple</code> containing an unhashable element

Lists are unhashable, so neither a list nor a tuple containing a list can be a key:

```
1 {[1]: "bad"}
```

py

TypeError: cannot use 'list' as a dict key

```
1 {(1, [2]): "bad"}
```

py

TypeError: cannot use 'tuple' as a dict key

You can always coerce a list to a tuple (but this is shallow),

```
1 {tuple([1]): "Okay"}
```

py

```
{(1,): 'Okay'}
```

```
1 {(1, tuple([2])): "Okay"}
```

py

```
{(1, (2,)): 'Okay'}
```

Lookup

Key lookup is via `[]`, if the key is missing a `KeyError` is raised. Alternatively, use `.get()` with a default. `in` checks for the presence of a key.

```
1 x = {1: "abc", "y": "hello", (1, 1): 3.14}
2 x[1]
```

'abc'

```
1 x[(1, 1)]
```

3.14

```
1 x["def"]
```

KeyError: 'def'

```
1 "y" in x
```

True

```
1 "hello" in x
```

False

```
1 print( x.get("def") )
```

None

```
1 x.get("def", 0)
```

0

Insert, replace, remove

Assigning to `d[key]` inserts or replaces a value for that key. `del` and `.pop()` remove key & value.

```
1 x = {1: "abc", "y": "hello"}
2 x["def"] = -1
3 x["y"] = "goodbye"
4 x
```

py

```
{1: 'abc', 'y': 'goodbye', 'def': -1}
```

```
1 del x[1]
2 x.pop("def")
```

py

```
-1
```

```
1 x["y"] = None
2 x
```

py

```
{'y': None}
```

```
1 y = list(abc = 123, def = 456)
2 y$ghi = -1
3 y$def = "goodbye"
4 str(y)
```

R

```
List of 3
 $ abc: num 123
 $ def: chr "goodbye"
 $ ghi: num -1
```

```
1 y$abc = NULL
2 str(y)
```

R

```
List of 2
 $ def: chr "goodbye"
 $ ghi: num -1
```

Methods & iteration

Iterating over a `dict` yields its keys - use `.items()` to get keys and values together.

```
1 x = {1: "abc", "y": "hello"}
```

py

```
1 x.keys()
```

py

```
dict_keys([1, 'y'])
```

```
1 x.values()
```

py

```
dict_values(['abc', 'hello'])
```

```
1 for k, v in x.items():  
2     print(k, "->", v)
```

py

```
1 -> abc  
y -> hello
```

```
1 x.items()
```

py

```
dict_items([(1, 'abc'), ('y', 'hello')])
```

```
1 x | {"y": "goodbye", "w": 0}
```

py

```
{1: 'abc', 'y': 'goodbye', 'w': 0}
```

```
1 x.update({"y": "goodbye", "w": 0})  
2 x
```

py

```
{1: 'abc', 'y': 'goodbye', 'w': 0}
```

`.keys()`, `.values()`, and `.items()` return dynamic views of the dictionary rather than copies - see the [docs](#).

Exercise 1

Using the record from last lecture, complete these tasks in Python:

```
1 person = {
2     "firstName": "John",
3     "lastName": "Smith",
4     "age": 25,
5     "address": {
6         "streetAddress": "21 2nd Street",
7         "city": "New York",
8         "state": "NY",
9         "postalCode": 10021
10    },
11    "phoneNumber": [
12        { "type": "home",
13          "number": "212 555-1239" },
14        { "type": "fax",
15          "number": "646 555-4567" }
16    ]
17 }
```

PY

- Extract the fax number.
- Add a "mobile" phone number and remove age.
- Write `phone(person, phone_type)` to return the requested number, or `None` if absent.

Sets

Sets

A **set** is a *mutable, unordered* collection of **unique** hashable elements - there are no positions, so the primary operation is membership testing with **in**.

```
1 x = {1, 2, 3, 4, 1, 2}
2 x
```

py

```
{1, 2, 3, 4}
```

```
1 set("mississippi")
```

py

```
{'p', 'i', 's', 'm'}
```

```
1 3 in x
```

py

```
True
```

```
1 x[0]
```

py

```
TypeError: 'set' object is not subscriptable
```

```
1 x.add(9)
2 x.discard(8)
3 x.update([7, 8])
4 x
```

py

```
{1, 2, 3, 4, 7, 8, 9}
```

```
1 x.remove(6)
```

py

```
KeyError: 6
```

```
1 {1, 2, [1, 2]}
```

py

```
TypeError: cannot use 'list' as a set element (u
```

`{}` creates an empty *dictionary*, an empty set must be written `set()` `remove()` raises a **KeyError** if the element is absent

Set operations

Sets support the usual mathematical operations. R has no set type - any vector can be treated as one via `unique()` and the set functions.

```
1 x = {1, 2, 3}
2 y = {2, 3, 4}
```

py

```
1 x = c(1, 2, 3, 1)
2 y = c(2, 3, 4)
```

R

```
1 x | y
```

py

```
1 union(x, y)
```

R

```
{1, 2, 3, 4}
```

```
[1] 1 2 3 4
```

```
1 x & y
```

py

```
1 intersect(x, y)
```

R

```
{2, 3}
```

```
[1] 2 3
```

```
1 x - y
```

py

```
1 setdiff(x, y)
```

R

```
{1}
```

```
[1] 1
```

```
1 1 in x
```

py

```
1 1 %in% x
```

R

```
True
```

```
[1] TRUE
```

Python's set methods (`x.union(y)`, `x.intersection(y)`, ...) accept any iterable, the operators require sets on both sides.

Algorithms & data structures

Big-O notation

Big-O notation describes the *complexity* of an algorithm - how the time (or memory) required grows with the size of the input n . Only the fastest growing term matters (constant factors and lower order terms are dropped), so two algorithms with the same Big-O can still differ greatly in practice.

Since performance depends on the data we usually quote *average* or *worst case* complexity, and *amortized* complexity when an occasional expensive step is paid for by many cheap ones (e.g. growing a list).

Complexity	Big-O
Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Quasilinear	$O(n \log n)$
Quadratic	$O(n^2)$
Exponential	$O(C^n)$

Under the hood

The containers in both languages are built from a handful of basic structures - knowing which is which explains what each container is good, and bad, at.

Structure	Layout	R	Python
Array	contiguous block of same-sized values	materialized numeric vector	contiguous numeric <code>ndarray</code>
Array of pointers	contiguous block of <i>references</i> to objects	list	<code>list</code> , <code>tuple</code>
Linked list	nodes that each point to their neighbors	pairlist	<code>deque</code> (linked blocks)
Hash table	array indexed by <code>hash(key)</code>	environment	<code>dict</code> , <code>set</code>

Vectors

Materialized numeric vectors in R and contiguous numeric NumPy arrays store fixed-size values together, giving $O(1)$ element access. NumPy views can have gaps or reversed strides.

```
1 object.size(numeric(1e6))
```

R

8000048 bytes

```
1 object.size(integer(1e6))
```

R

4000048 bytes

```
1 np.zeros(1_000_000).nbytes
```

py

8000000

```
1 np.zeros(1_000_000, dtype="int64").nbytes
```

py

8000000

```
1 np.zeros(1_000_000, dtype="int32").nbytes
```

py

4000000

R doubles use 8 bytes per element; integers use 4. `object.size()` includes object overhead; NumPy's `.nbytes` counts

Growing a vector

Repeatedly appending with `c(x, i)` copies the existing elements each time, giving $O(n^2)$ work overall.

```
1 n = 5e4
2 x = c()
3 system.time(
4   for (i in seq_len(n)) x = c(x, i)
5 )
```

user	system	elapsed
0.997	0.016	1.013

Indexed growth can reuse spare capacity, but still needs occasional copying. Preallocation avoids repeated growth.

```
1 x = numeric()
2 system.time(
3   for (i in seq_len(n)) x[i] = i
4 )
```

user	system	elapsed
0.004	0.000	0.004

```
1 x = numeric(n)
2 system.time(
3   for (i in seq_len(n)) x[i] = i
4 )
```

user	system	elapsed
0.002	0.000	0.002

Array (Vector)



<https://visualgo.net/en/array> - Materialized numeric R vectors and contiguous numeric NumPy arrays store same-typed values together. A Python `list` is a contiguous array of *pointers* to objects, which is why it can be heterogeneous and

Generic vectors

R and Python lists store references to objects. Elements can have different types and sizes, while indexing remains $O(1)$. References and object headers add memory overhead.

- Numeric vector - values stored contiguously:

```
[ 1.0 | 2.0 | 3.0 ]
```

- List - *references* stored contiguously

```
[ P | P | P ]  
  ↓   ↓   ↓  
  1  "abc" TRUE
```

deques

A **deque** (double-ended queue) supports $O(1)$ additions and removals at either end. CPython stores it as a doubly linked chain of fixed-size blocks. Indexing is $O(1)$ near either end and $O(n)$ in the middle.

```
1 from collections import deque
2 x = deque(range(3))
3 x.appendleft(-1)
4 x
```

py

```
deque([-1, 0, 1, 2])
```

```
1 x.popleft()
```

py

```
-1
```

```
1 x
```

py

```
deque([0, 1, 2])
```

```
1 x = deque(range(3), maxlen=4)
2 x.append(10)
3 x.append(11)
4 x
```

py

```
deque([1, 2, 10, 11], maxlen=4)
```

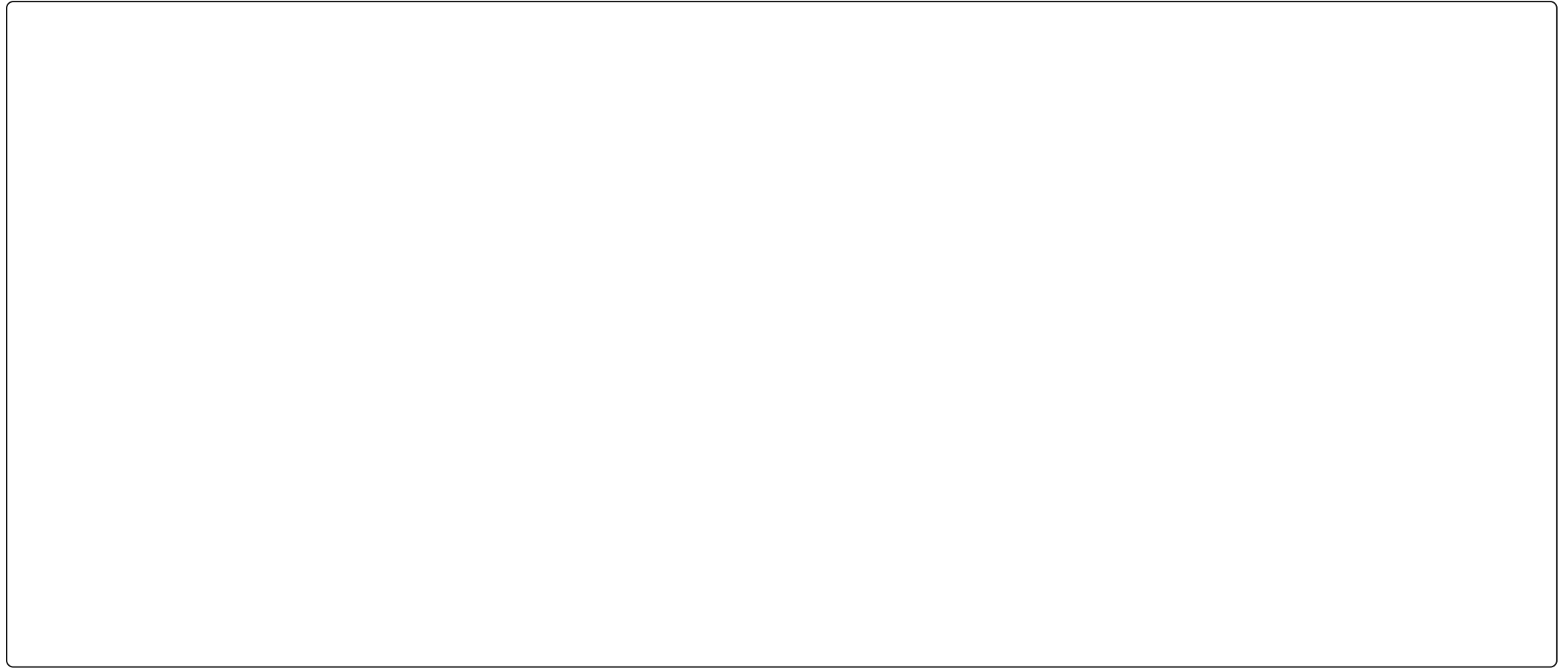
```
1 x.appendleft(-1)
2 x
```

py

```
deque([-1, 1, 2, 10], maxlen=4)
```

With **maxlen** set, adding to a full deque drops an element from the opposite end - a rolling window of the last n items.

Linked list



Hashing

A hash table is an array of *buckets* - a key's `hash()` picks its bucket, so lookup, insertion and deletion are $O(1)$ on average with no scanning. Two keys landing in the same bucket is a *collision*, and the table is resized as it fills to keep collisions rare.

```
1 hash(1), hash(1.0), hash(True)
```

py

```
(1, 1, 1)
```

```
1 hash(2.5)
```

py

```
1152921504606846978
```

```
1 hash((1, 2))
```

py

```
-3550055125485641917
```

```
1 keys = (1, 9, 17, 2.5)
2 [hash(k) % 8 for k in keys]
```

py

```
[1, 1, 1, 2]
```

```
1 h = new.env(size = 8L)
2 for (k in letters[1:5]) h[[k]] = k
3 str(env.profile(h))
```

R

List of 3

```
$ size      : int 8
$ nchains:  int 5
$ counts   : int [1:8] 0 1 1 1 1 1 0 0
```

```
1 for (k in letters[6:20]) h[[k]] = k
2 env.profile(h)$size
```

R

```
[1] 25
```

Equal keys must hash equally, which is why `1`, `1.0`, and `True` are the same `dict` key. String hashes are randomized per process, so `hash("abc")` (and the iteration order of a set of strings) changes between runs. `env.profile()` reports the

Hash table



<https://visualgo.net/en/hashtable> - Python's `dict` and `set` are hash tables, as are R environments. R's `match()`, `%in%`,
Sta 523 - Fall 2026

Operation costs

Operation	<code>list</code>	<code>deque</code>	<code>dict</code>	<code>set</code>
Copy	<code>.copy(): O(n)</code>	<code>.copy(): O(n)</code>	<code>.copy(): O(n)</code>	<code>.copy(): O(n)</code>
Get / set by index or key	<code>s[i]: O(1)</code>	<code>s[i]: O(n)</code>	<code>d[k]: O(1)</code>	—
Append / add	<code>.append(v): O(1)*</code>	<code>.append(v): O(1)</code>	<code>d[k] = v: O(1)*</code>	<code>.add(v): O(1)*</code>
Pop from end / remove	<code>.pop(): O(1)*</code>	<code>.pop(): O(1)</code>	<code>.pop(k): O(1)</code>	<code>.remove(v): O(1)</code>
Insert at front	<code>.insert(0, v): O(n)</code>	<code>.appendleft(v): O(1)</code>	—	—
Remove from front	<code>.pop(0): O(n)</code>	<code>.popleft(): O(1)</code>	—	—
Insert / delete in middle	<code>O(n)</code>	<code>O(n)</code>	—	—
Membership	<code>v in s: O(n)</code>	<code>v in s: O(n)</code>	<code>k in d: O(1)</code>	<code>v in s: O(1)</code>

CPython costs for n elements.

*Amortized - list append and end pop, and hash table insertion, occasionally trigger a resize. Deque indexing is $O(1)$ near either end. Hash table costs are averages that assume constant-cost hashing and equality, an individual operation can

Exercise 2

For each of the following, suggest a data structure in Python and R. Explain which operations your choice makes efficient, and state any assumptions.

- A collection of 100 integers used repeatedly in elementwise arithmetic.
- A queue (first in, first out) of customer records.
- A stack (first in, last out) of customer records.
- A count of word occurrences within a document.
- The heights of the bars in a histogram with even bin widths.
- Checking whether each of a million words is in a list of 50 stop words.

Measuring vector growth

Reasoning about complexity tells you what to expect, measuring confirms it

Code Linear Log

```
1 grow_concat = function(n) {  
2   x = c()  
3   for (i in seq_len(n)) x = c(x, i)  
4 }  
5 grow_index = function(n) {  
6   x = numeric()  
7   for (i in seq_len(n)) x[i] = i  
8 }  
9 grow_prealloc = function(n) {  
10  x = numeric(n)  
11  for (i in seq_len(n)) x[i] = i  
12 }
```

R

```
1 res = bench::press(  
2   n = c(1e3, 2.5e3, 5e3, 1e4, 2.5e4, 5e4),  
3   bench::mark(  
4     concat = grow_concat(n),  
5     index = grow_index(n),  
6     prealloc = grow_prealloc(n),  
7     check = FALSE  
8   )  
9 )
```

R

`bench::mark()` repeats each expression and reports the distribution of timings, `bench::press()` runs it over a grid of

Comparing R & Python

Container summary

Python	Properties	R
<code>list</code>	ordered, mutable, heterogeneous	<code>list()</code>
<code>tuple</code>	ordered, immutable, heterogeneous	<code>list()</code> or atomic vector (copy on modify)
<code>dict</code>	key lookup, mutable, insertion ordered	named <code>list()</code> , <code>new.env()</code> for hashing
<code>set</code>	unique, unordered, mutable	vector + <code>unique()</code> , <code>union()</code> , <code>%in%</code> , ...
<code>deque</code>	fast at both ends	none (use a list or vector)
<code>range</code>	lazy integer sequence, immutable	<code>seq_len()</code> , <code>:</code> (often stored compactly via ALTREP)
NumPy <code>ndarray</code>	homogeneous, vectorized	atomic vector, matrix, array

Takeaways

- **Tuples** are immutable sequences—use them for fixed groups of values and unpacking.
- **Dictionaries** map hashable keys to values, with $O(1)$ lookup on average. R offers named lists and hash-based environments.
- **Sets** provide $O(1)$ membership on average and set algebra. In R, use `%in%`, `union()`, `intersect()`, and `setdiff()`.
- **Shallow copies** share nested objects. Use `deepcopy()` when nested objects need independent mutation.
- **Mutable defaults** are shared between calls. Use `None` and create the object inside the function.
- Choose containers for their operations: deques for queues, sets for membership, and preallocated R vectors when the size is known. Measure when performance matters.